

Lab 1

2. Java Swing API – Input Handling

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, January 19th, 2026 | ER 3120



University of Windsor

Part I

Install the Development Environment





Install the JDK



Step 1: Installing the JDK

- If you are deciding to use Eclipse, you can skip this step. If you are using VS Code, download the JDK from Oracle.
- Eclipse comes with its own ECJ (Eclipse Compiler for Java) since its an IDE.

JDK Development Kit 21.0.2 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee.

Linux macOS **Windows**

Product/file description	File size	Download
x64 Compressed Archive	185.52 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	163.91 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	162.07MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

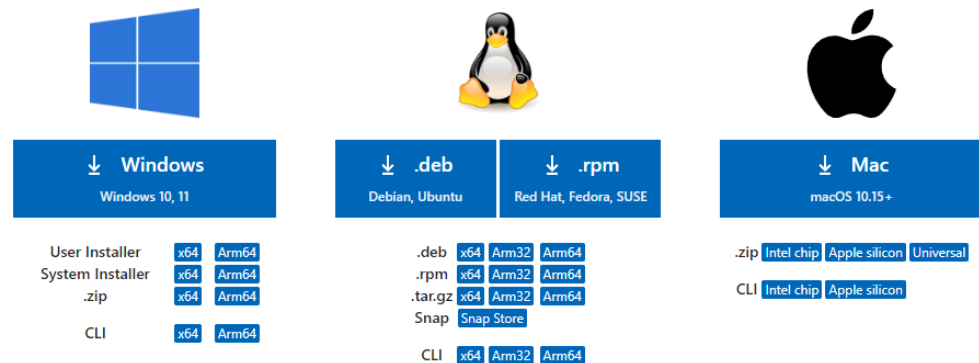


Step 2: Download Eclipse / VS Code

- Depending on your preference, download either Eclipse or Visual Studio Code

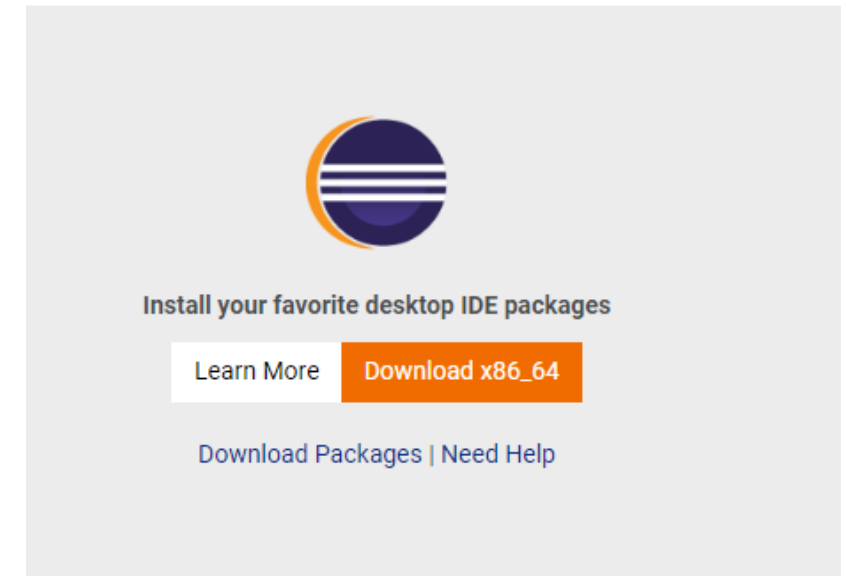
Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



The Visual Studio Code download page features three main sections for different operating systems: Windows, Linux, and Mac. Each section includes a download icon, the operating system name, and a list of available installation methods with their respective architectures.

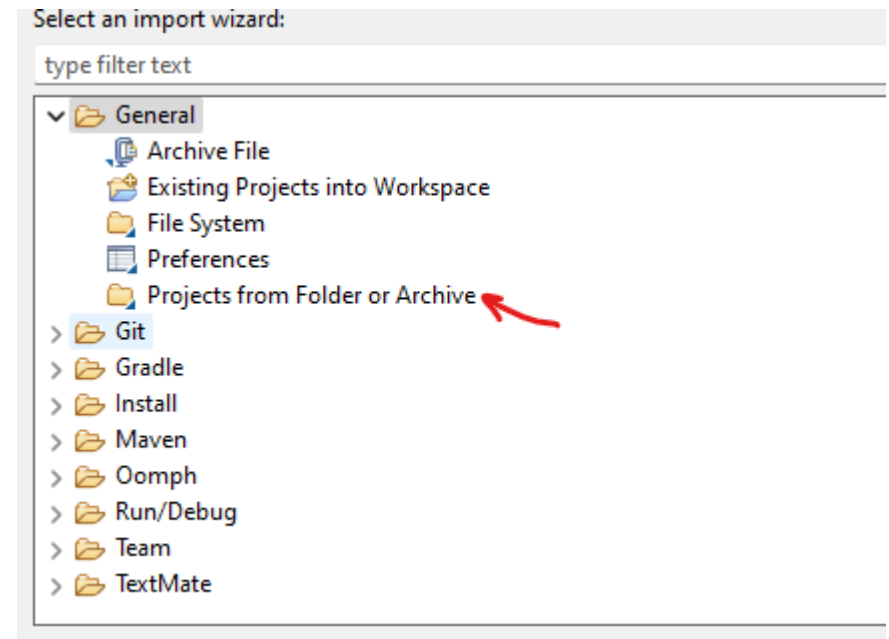
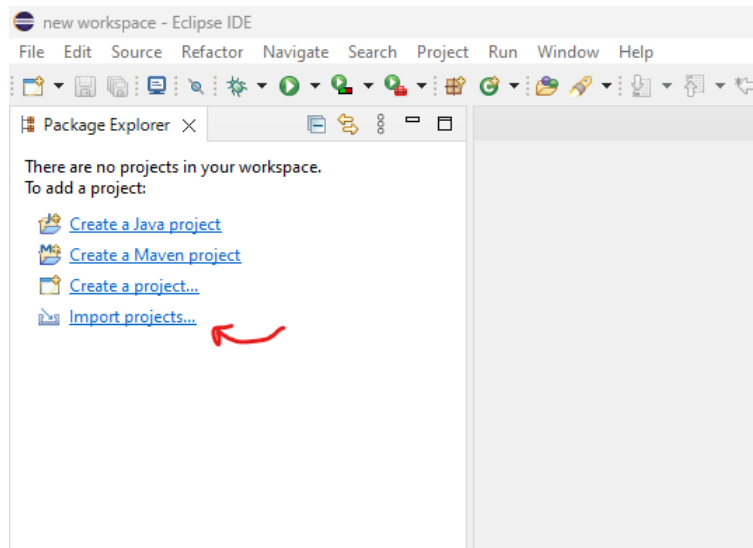
Operating System	Installation Method	Architectures
Windows (Windows 10, 11)	User Installer	x64, Arm64
	System Installer	x64, Arm64
	.zip	x64, Arm64
	CLI	x64, Arm64
Linux (Debian, Ubuntu, Red Hat, Fedora, SUSE)	.deb	x64, Arm32, Arm64
		x64, Arm32, Arm64
	.rpm	x64, Arm32, Arm64
		x64, Arm32, Arm64
	.tar.gz	x64, Arm32, Arm64
	Snap	Snap Store
	CLI	x64, Arm32, Arm64
	Mac (macOS 10.15+)	.zip
CLI		Intel chip, Apple silicon



The Eclipse IDE download page features a large circular logo at the top. Below it, the text "Install your favorite desktop IDE packages" is displayed. There are two buttons: "Learn More" and "Download x86_64". At the bottom, there is a link "Download Packages | Need Help".

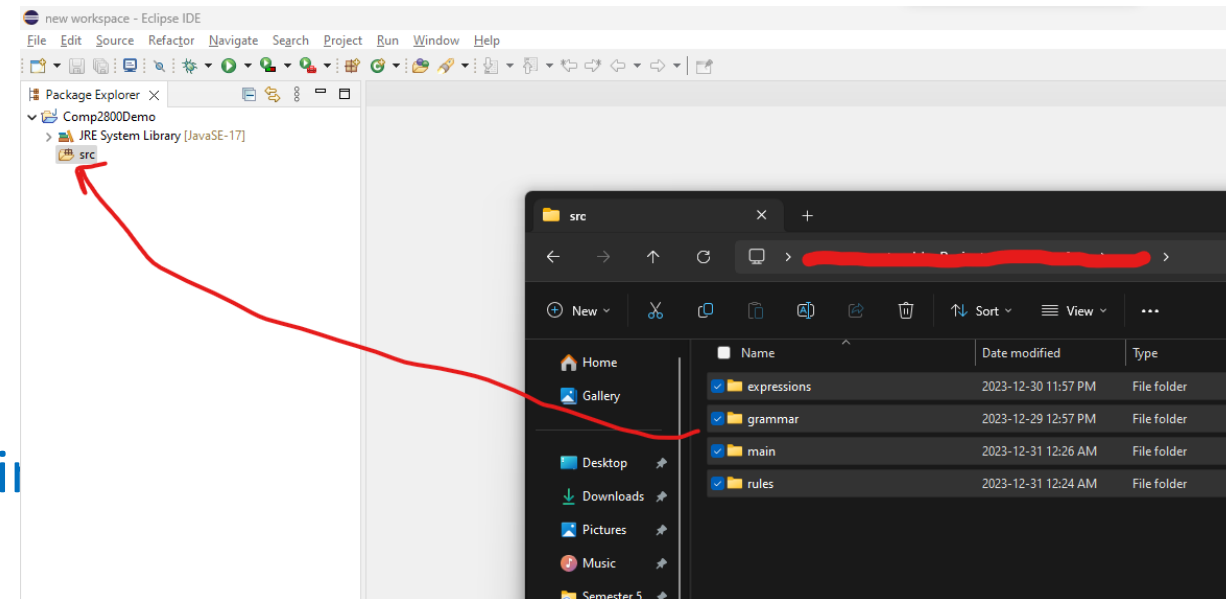
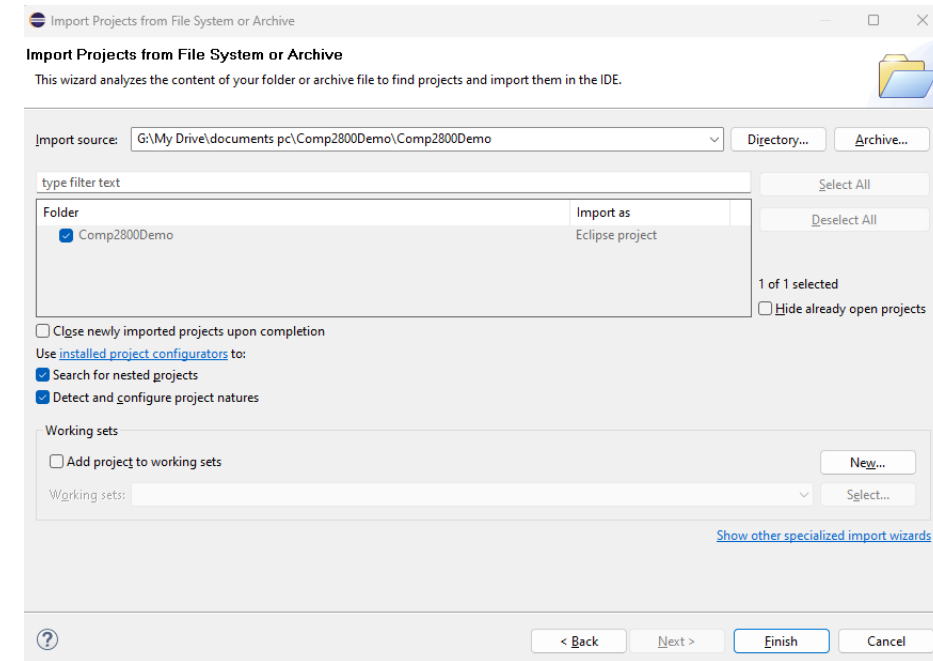
Adding existing code into Eclipse

- This option allows you to add an existing Eclipse project to your workspace. You can add an existing project to your new workspace using this option. Then select Projects from Folder or Archive.



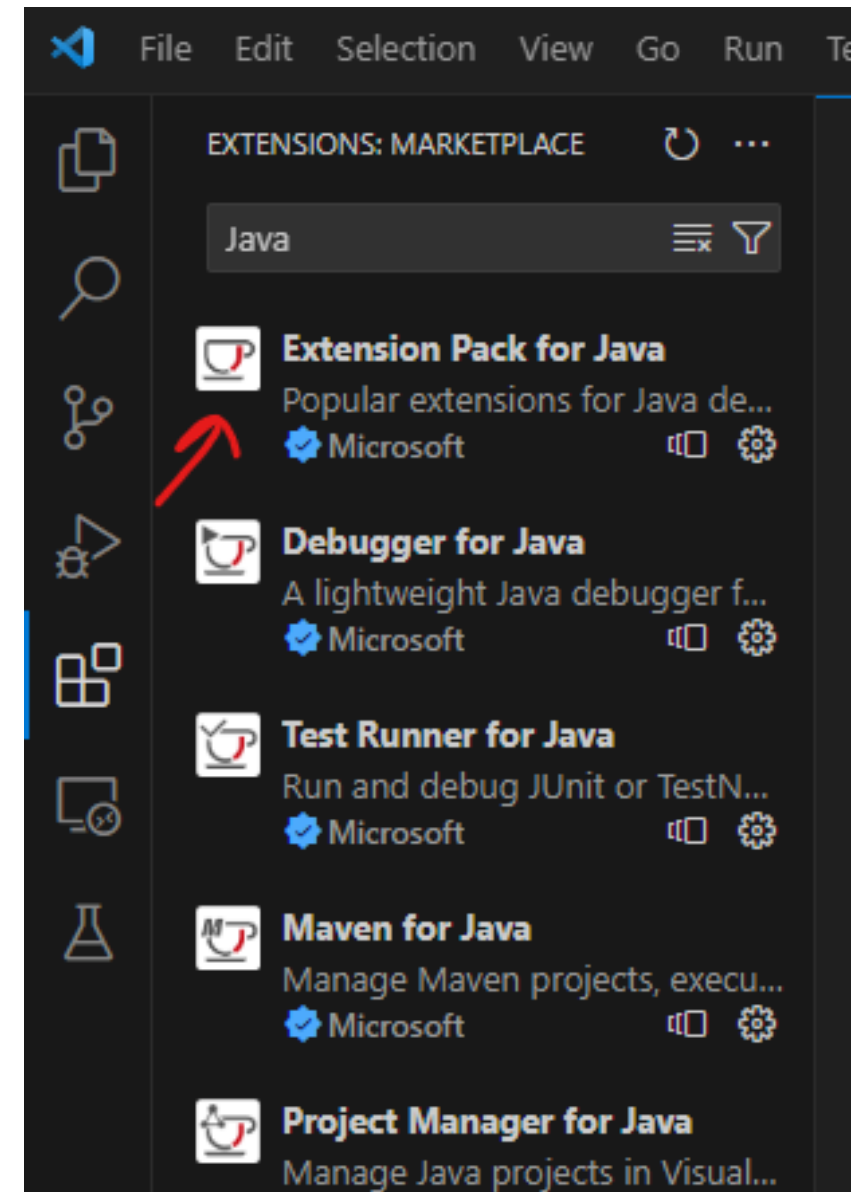
Add existing code to Eclipse (cont.)

- Here you can select the directory where your project exists. After clicking Finish, your project will be imported. This will take care of everything including any external libraries that were referenced in your existing project.
- If you only want to import the source code located in some folder or zip file, you can simply copy the packages or code files and paste them into your Project's src folder. If this source code uses external libraries, you can reference those jars in the project settings.



Import existing code into VS Code

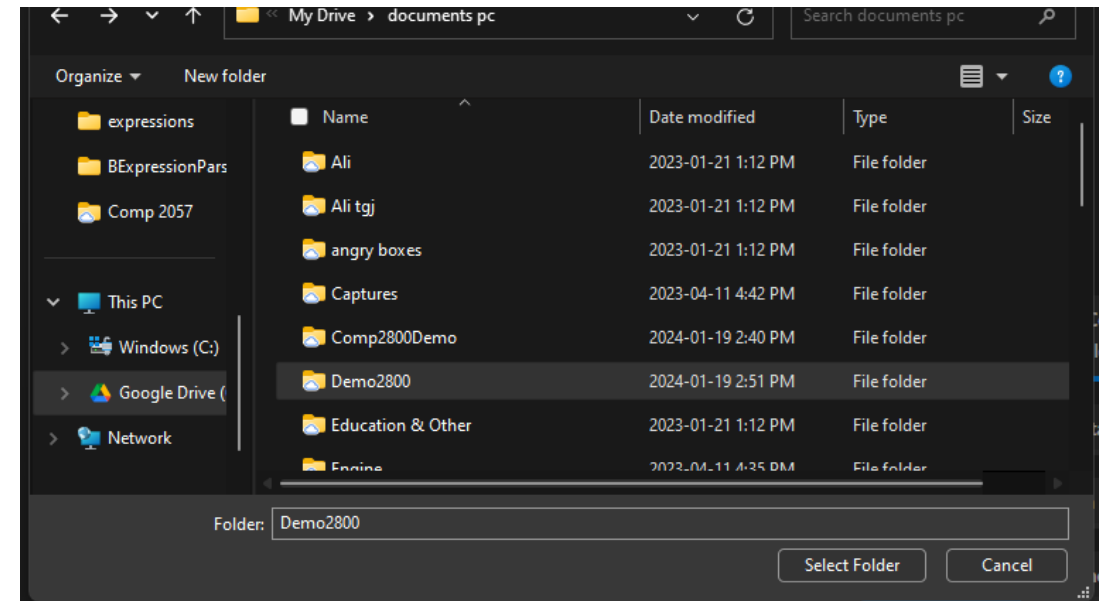
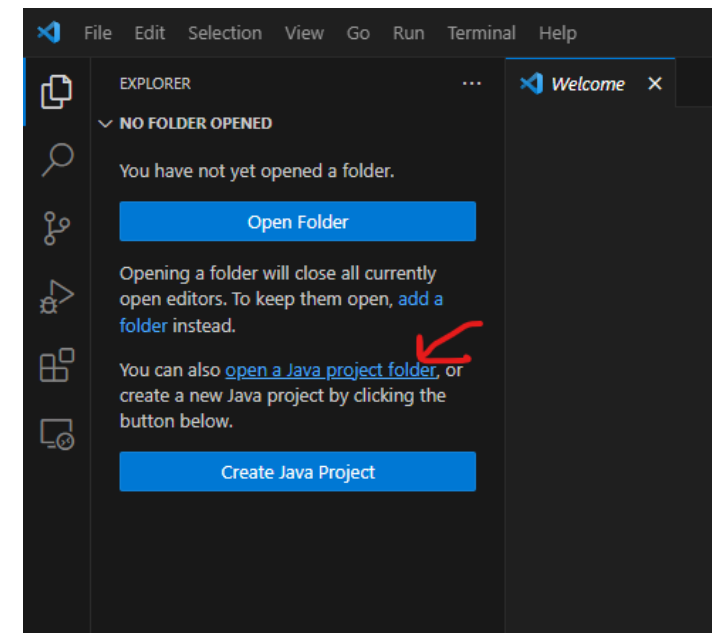
- First Run VS Code and download the Java Extension Pack.



Add existing code to VS Code (cont.)

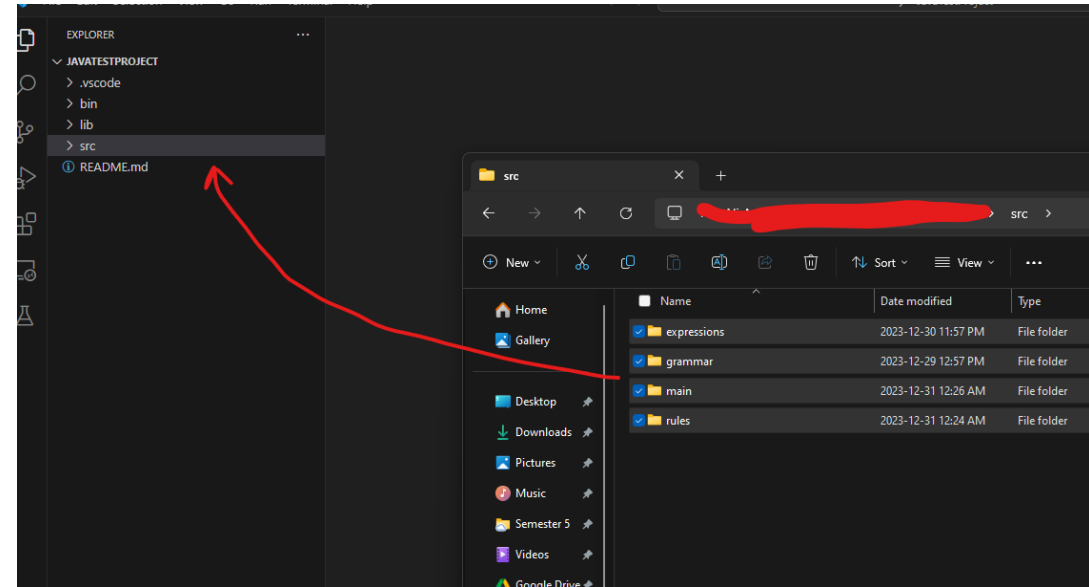
- Select the Open a Java project folder option in VS Code and locate your existing project.
- Click *select folder* and you should be set.

This will take care of all the project settings, and external libraries that may have been referenced in your existing project.



Adding existing code to VS Code (cont.)

- If you only want to bring the source code of an existing project into your new project, you can simply copy and paste the existing source code packages into the src folder in VS Code project.
- If your existing code references external libraries, you can copy those jar files into the lib folder. The bin folder contains the compiled bytecode files (.class files).

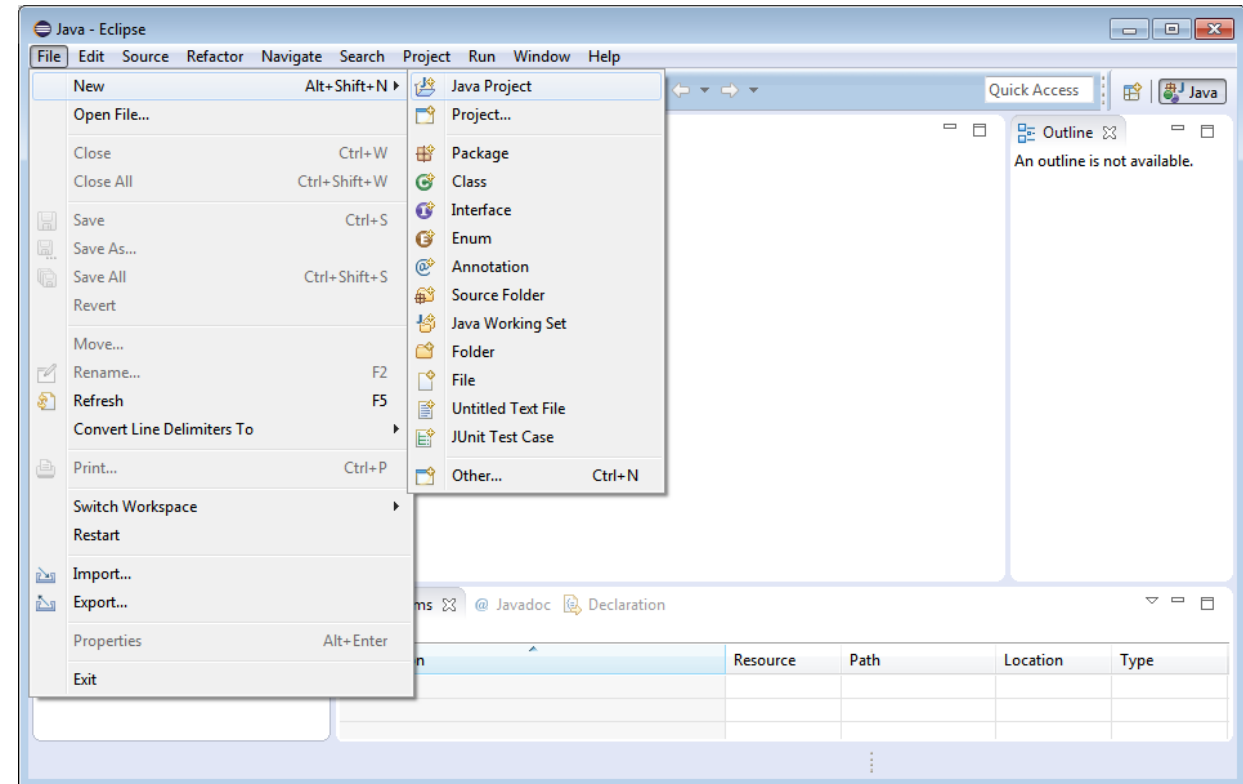


Create a Java Project



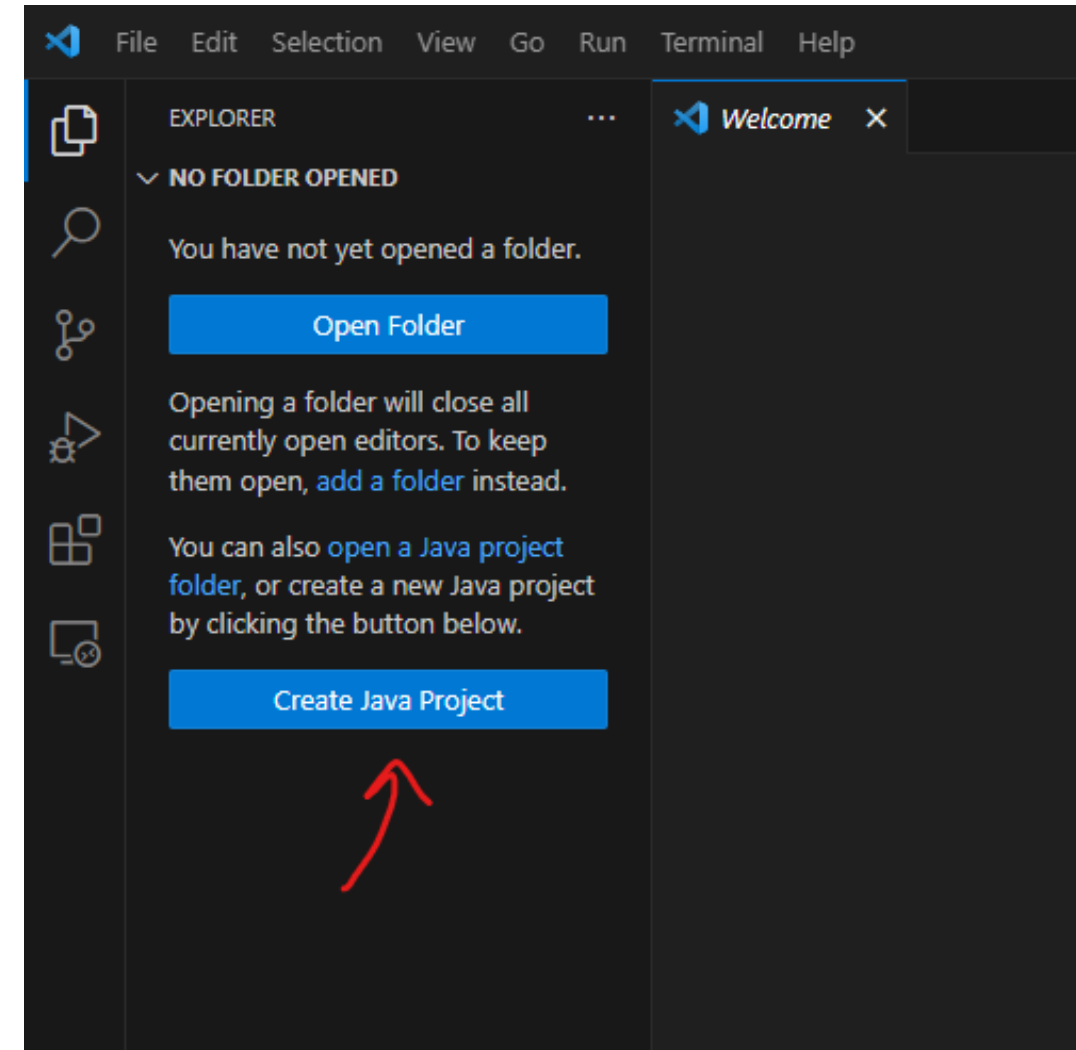
Creating Java Project (Eclipse)

- Run Eclipse and choose a workspace folder.
- Then create a new Java Project.



Creating Java Project (VS Code)

- Create a new Java Project



Part II

Coding for the Lab



Producing the Java code for the Lab



Creating a Window

- Import the Swing and AWT package.
- Instantiate the JFrame class to create a window.
 - JFrame represents a Window in the relative operating system.
 - Setting the default close operation to EXIT_ON_CLOSE allows the program to stop executing when the JFrame is closed.
 - Setting its location relative to null, allows it to be centered.

```
import javax.swing.JFrame;

public class Demo {

    public static void main(String[] args) {
        JFrame frame = new JFrame(title:"Comp 2800 Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(width:800, height:600);
        frame.setLocationRelativeTo(c:null);
        frame.setVisible(b:true);
    }
}
```



Creating a Button

- We need to create a JPanel object which will hold all our buttons and other UI elements.
- It is advised to use a JPanel for adding elements to the JFrame instead of directly using the JFrame.
- Now we have created a JButton and added it to the panel.

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Demo {

    public static void main(String[] args) {
        JFrame frame = new JFrame(title:"Comp 2800 Demo");
        frame.setSize(width:400, height:300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(c:null);

        JPanel panel = new JPanel();
        JButton button = new JButton(text:"Click Me");
        panel.add(button);

        frame.add(panel);
        frame.setVisible(b:true);
    }
}
```

Performing an action on button click.

- We can add an action listener interface to the button and dictate what should happen should the button be clicked.
- In this case it will print “clicked!”

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Demo {

    public static void main(String[] args) {
        JFrame frame = new JFrame("Comp 2800 Demo");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);

        JPanel panel = new JPanel();
        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener(){

            public void actionPerformed(ActionEvent e){
                System.out.println("clicked!");
            }

        });

        panel.add(button);

        frame.add(panel);
        frame.setVisible(true);
    }
}
```

Getting mouse input

- We can get mouse input in a similar manner. The specific mouse button that is pressed can be obtained through the MouseEvent parameter.
- There are five methods in the MouseListener interface:
 - void mouseClicked(MouseEvent e): Invoked when the mouse button is clicked (pressed and released) on a component.
 - void mousePressed(MouseEvent e): Invoked when a mouse button is pressed on a component.
 - void mouseReleased(MouseEvent e): Invoked when a mouse button is released on a component.
 - void mouseEntered(MouseEvent e): Invoked when the mouse enters a component.
 - void mouseExited(MouseEvent e): Invoked when the mouse exits a component.
- We are only using the first one in this example.

```
import java.awt.event.ActionEvent;
import java.awt.event.MouseEvent;
import java.awt.event.ActionListener;
import java.awt.event.MouseAdapter;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class Demo {

    public static void main(String[] args) {
        JFrame frame = new JFrame("Comp 2800 Demo");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);

        JPanel panel = new JPanel();
        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button Clicked!");
            }
        });

        panel.addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                System.out.println("Mouse pressed!");
            }
        });

        panel.add(button);
        frame.getContentPane().add(panel);
        frame.setVisible(true);
    }
}
```

Getting Mouse motion input

- You can obtain mouse motion input by following similar pattern. The MouseMotionListener interface contains 2 methods:
 - void mouseDragged(MouseEvent e): Invoked when a mouse button is pressed on a component and then dragged.
 - void mouseMoved(MouseEvent e): Invoked when the mouse cursor has been moved onto a component but no buttons have been pressed.

```
public class Demo {  
  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("Comp 2800 Demo");  
        frame.setSize(400, 300);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setLocationRelativeTo(null);  
  
        JPanel panel = new JPanel();  
        JButton button = new JButton("Click Me");  
  
        button.addActionListener(new ActionListener() {  
            @Override  
            public void actionPerformed(ActionEvent e) {  
                System.out.println("Button Clicked!");  
            }  
        });  
  
        panel.addMouseListener(new MouseAdapter() {  
            @Override  
            public void mousePressed(MouseEvent e) {  
                System.out.println("Mouse pressed!");  
            }  
        });  
  
        panel.addMouseMotionListener(new MouseMotionAdapter() {  
            @Override  
            public void mouseMoved(MouseEvent e) {  
                System.out.println("Mouse moved - X: " + e.getX() + ", Y: " + e.getY());  
            }  
        });  
  
        panel.add(button);  
        frame.getContentPane().add(panel);  
        frame.setVisible(true);  
    }  
}
```

Getting Keyboard input

- Key input is also implemented similarly. You can get the specific key using the KeyEvent parameter. There are 3 methods in the KeyListener interface:
 - void keyPressed(KeyEvent e) - Invoked when a key is pressed down. This event occurs once for each key press.
 - void keyReleased(KeyEvent e) - Invoked when a key is released after being pressed. This event occurs once for each key release.
 - void keyTyped(KeyEvent e) - Invoked when a key is typed. This event occurs when a key press is followed by a key release.

```
import java.awt.event.*;
import javax.swing.*;

public class Demo {

    public static void main(String[] args) {
        JFrame frame = new JFrame("Comp 2800 Demo");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);

        JPanel panel = new JPanel();
        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button Clicked!");
            }
        });

        panel.addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                System.out.println("Mouse pressed!");
            }
        });

        panel.addMouseMotionListener(new MouseMotionAdapter() {
            @Override
            public void mouseMoved(MouseEvent e) {
                System.out.println("Mouse moved - X: " + e.getX() + ", Y: " + e.getY());
            }
        });

        panel.addKeyListener(new KeyAdapter() {
            public void keyPressed(KeyEvent e) {
                System.out.println("s");
                if (e.getKeyCode() == KeyEvent.VK_X) {
                    System.out.println("X pressed!");
                } else if (e.getKeyCode() == KeyEvent.VK_Y) {
                    System.out.println("Y pressed!");
                }
            }
        });

        panel.setFocusable(true);
        panel.add(button);
        frame.getContentPane().add(panel);
        frame.setVisible(true);
    }
}
```

Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor